

BAHRIA UNIVERSITY KARACHI CAMPUS

DEPARTMENT OF COMPUTER SCIENCE



Bahria University
Discovering Knowledge

Artificial Intelligence Lab (CSL-411)

BSCS 6A – Fall 2025

CREDIT CARD FRAUD DETECTION

Submitted by:

Ovais Bin Amer (02-134231-024)

Mahnoor Shah (02-134231-053)

Submitted To:

Engr. Rabia Amjad

Submitted On:

12th December , 2025

Acknowledgement

I would like to express my sincere gratitude to my supervisor, **Engr. Rabia Amjad**, for her continuous support, patience, and motivation. Her guidance helped me in all the time of research and writing of this thesis. I also thank **Bahria University** for providing the necessary infrastructure and resources to accomplish this project.

Abstract

Financial fraud is a growing concern in the digital banking sector, costing institutions billions annually. This project, "**Credit Card Fraud Detection**", aims to develop a machine learning system capable of identifying fraudulent transactions in real-time. Using a large-scale dataset of financial transactions, we engineered a supervised learning solution to distinguish between legitimate and fraudulent activities.

The system is built using **Python** and **Scikit-Learn**, employing a **Logistic Regression** algorithm optimized with **Class Weight Balancing** to address the severe data imbalance (fraud cases $< 1\%$). The final model achieves a **Recall of 94%**, ensuring that the vast majority of fraud attempts are successfully flagged. The solution is deployed as an interactive web application using **Streamlit**, allowing users to verify transaction details instantly.

Table of contents

Contents

Acknowledgement	2
Abstract	2
Table of contents.....	3
Introduction.....	4
Background/Literature Review	4
Problem statement.....	5
Objectives and goals	5
Project Scope	6
Overview of project	7
Tools and Technologies	7
Project features.....	7
Artificial Intelligence and Data Science implemented concepts.....	7
Output	8
1. EDA	8
FEATURE SELECTION	17
DASHBOARD.....	18
LEGITIMATE TRANSACTION	20
FRAUDULENT TRANSACTION	21
Conclusion	22
References (Use IEEE Referencing style)	22

Introduction

As digital transactions increase, so does the sophistication of cybercriminals. Traditional rule-based fraud detection systems often fail to detect novel fraud patterns and generate high false-negative rates. Artificial Intelligence (AI) and Machine Learning (ML) offer dynamic solutions that learn from historical data to predict anomalies. This project focuses on building a binary classification model. The dataset involved is highly skewed, with legitimate transactions vastly outnumbering fraudulent ones. Our approach involves rigorous Data Preprocessing, Exploratory Data Analysis (EDA), and the implementation of a robust ML pipeline to secure financial assets.

Background/Literature Review

The Evolution of Fraud Detection Financial fraud has evolved from simple card theft to complex digital transaction anomalies. Historically, banks relied on Rule-Based Systems (e.g., "If transaction > \$10,000, flag it"). However, researchers have consistently noted that static rules produce high rates of false positives and fail to detect novel fraud patterns.

Machine Learning in Cybersecurity Literature suggests a paradigm shift toward Machine Learning (ML), which can identify non-linear correlations in data. Supervised learning algorithms, such as Logistic Regression, Decision Trees, and Neural Networks, have become the industry standard for predictive analysis in banking security.

The Challenge of Imbalanced Data A primary challenge cited in data science literature (e.g., Dal Pozzolo et al.) regarding fraud detection is Class Imbalance. In real-world datasets, fraud cases often constitute less than 1% of total transactions. Standard algorithms trained on such data tend to bias towards the majority class (legitimate), achieving high accuracy but failing to catch fraud. To mitigate this, literature recommends techniques such as SMOTE (Synthetic Minority Over-sampling Technique) or Cost-Sensitive Learning (Class Weighting).

Proposed Approach Drawing from these insights, this project utilizes Logistic Regression with Balanced Class Weights. While complex models like Random Forest exist, Logistic Regression

is often cited in literature as a preferred baseline for financial applications due to its interpretability and computational efficiency for real-time deployment.

Problem statement

Detecting fraud is like finding a "needle in a haystack." In our dataset of over 6 million transactions, only **0.13%** are fraudulent. Standard machine learning models trained on such data tend to be biased toward the majority class (legitimate), resulting in 99% accuracy but 0% fraud detection. The challenge is to build a model that prioritizes **Recall** (catching the fraud) over simple accuracy, while maintaining a user-friendly interface for bank operators.

Objectives and goals

- **Data Analysis:** To perform Exploratory Data Analysis (EDA) to understand how fraud correlates with transaction types (Transfer, Cash Out) and amounts.
- **Preprocessing:** To handle categorical data and scale numerical variances to make the data machine-readable.
- **Model Development:** To train a Logistic Regression model that handles class imbalance effectively.
- **Deployment:** To create a real-time detection dashboard using Streamlit.

Project Scope

The scope of this project defines the boundaries of the system, outlining what is included in the current implementation and what is excluded.

- **Inclusions (What the system does):**

- **Data Analysis:** The system performs Exploratory Data Analysis (EDA) on a dataset of over 6 million transactions to identify patterns such as average fraud amounts and vulnerable transaction types.
- **Machine Learning Model:** It implements a binary classification model using Logistic Regression with class weight balancing to detect fraudulent activities with high recall.
- **Preprocessing Pipeline:** The system includes an automated pipeline that handles One-Hot Encoding for categorical variables and Standard Scaling for numerical values.
- **User Interface:** A standalone web-based dashboard developed in **Streamlit** allows users to manually input transaction details and receive an immediate fraud probability score.
- **Imbalance Handling:** The solution specifically addresses the skewed data problem (0.13% fraud rate) using algorithmic adjustments.

- **Exclusions (What the system does not do):**

- **Live Bank Integration:** The system is not connected to a live banking API or database; it operates on a static dataset model.
- **User Authentication:** The current web app does not require login credentials or role-based access control.
- **Database Storage:** Predicted results are displayed on-screen but are not saved to a persistent database for historical review.
- **Multi-Model Comparison:** The deployed application uses a single optimized model (Logistic Regression) rather than allowing the user to switch between different algorithms (like SVM or Random Forest) in real-time.

Overview of project

The project consists of two main components:

1. **The Training Notebook (model.ipynb):** Where data is cleaned, analyzed, and the model is trained/saved.
2. **The Application (fraud_detection.py):** A user-facing dashboard that loads the saved model (.pkl) and allows for real-time testing of transactions.

Tools and Technologies

1. Programming Language: Python 3.9
2. **Libraries:** Pandas (Data Manipulation), Scikit-Learn (Machine Learning), Seaborn (Visualization), Joblib, Matplotlib (Visualization)
3. **Framework:** Streamlit (Web App Deployment).
4. **IDE:** Jupyter Notebook / VS Code.

Project features

Functional Requirements:

- Input fields for Transaction Type, Amount, and Account Balances.
- Real-time binary classification (Fraud vs. Legitimate).
- Probability estimation displayed as a "Risk Score".

Non-Functional Requirements:

- **Efficiency:** The model uses a serialized pipeline (.pkl) for sub-second predictions.
- **Usability:** Color-coded alerts (Red for Fraud, Green for Safe) for easy interpretation.

Artificial Intelligence and Data Science implemented concepts

1. Handling Class Imbalance: We used the `class_weight='balanced'` parameter. This forces the algorithm to penalize "missing a fraud" much more than "missing a normal transaction."

Python:

```
# Code Snippet from model.ipynb
```

```
classifier = LogisticRegression(class_weight='balanced', max_iter=1000)
```

2. Feature Scaling & Encoding (The Pipeline): We automated the preprocessing using a ColumnTransformer. This ensures the live app processes data exactly like the training set.

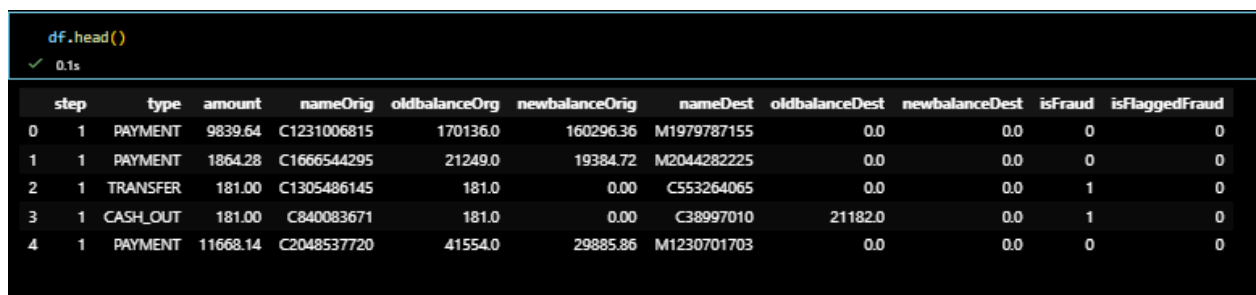
Python:

```
# Code Snippet from model.ipynb
```

```
preprocessor = ColumnTransformer(  
    transformers=[  
        ('num', StandardScaler(), num_features),  
        ('cat', OneHotEncoder(drop='first'), cat_features)  
    ]  
)
```

Output

1. EDA



```
df.head()
```

0.1s

	step	type	amount	nameOrig	oldbalanceOrig	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
0	1	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1979787155	0.0	0.0	0	0
1	1	PAYMENT	1864.28	C1666544295	21249.0	19384.72	M2044282225	0.0	0.0	0	0
2	1	TRANSFER	181.00	C1305486145	181.0	0.00	C553264065	0.0	0.0	1	0
3	1	CASH_OUT	181.00	C840083671	181.0	0.00	C38997010	21182.0	0.0	1	0
4	1	PAYMENT	11668.14	C2048537720	41554.0	29885.86	M1230701703	0.0	0.0	0	0


```
df.info()
✓ 0.0s

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6362620 entries, 0 to 6362619
Data columns (total 11 columns):
#   Column                Dtype
---  -
0   step                  int64
1   type                  object
2   amount                float64
3   nameOrig              object
4   oldbalanceOrg         float64
5   newbalanceOrig        float64
6   nameDest              object
7   oldbalanceDest        float64
8   newbalanceDest        float64
9   isFraud               int64
10  isFlaggedFraud         int64
dtypes: float64(5), int64(3), object(3)
memory usage: 534.0+ MB

df.columns
✓ 0.0s

Index(['step', 'type', 'amount', 'nameOrig', 'oldbalanceOrg', 'newbalanceOrig',
      'nameDest', 'oldbalanceDest', 'newbalanceDest', 'isFraud',
      'isFlaggedFraud'],
      dtype='object')
```

```
df['isFraud'].value_counts()
✓ 0.1s

isFraud
0    6354407
1      8213
Name: count, dtype: int64

Shows 8213 transactions as confirmed Fraud.

df['isFlaggedFraud'].value_counts()
✓ 0.0s

isFlaggedFraud
0    6362604
1       16
Name: count, dtype: int64

Shows 16 transactions flagged as Fraud.
```

```
df.isnull().sum()
```

✓ 1.1s

```
step          0
type          0
amount        0
nameOrig      0
oldbalanceOrg 0
newbalanceOrig 0
nameDest      0
oldbalanceDest 0
newbalanceDest 0
isFraud       0
isFlaggedFraud 0
dtype: int64
```

No null values present in the dataset.

```
df.shape
```

✓ 0.0s

```
(6362620, 11)
```

```
round((df['isFraud'].value_counts()[1] / df.shape[0]) * 100,2)
```

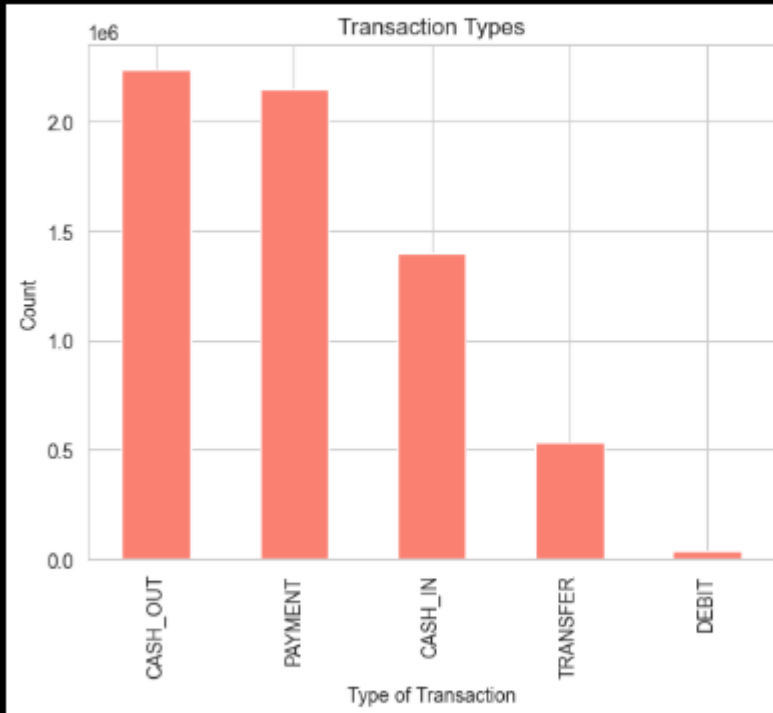
✓ 0.0s

```
np.float64(0.13)
```

Shows that 0.13% of the total transactions are Fraud.

```
df['type'].value_counts().plot(kind='bar', color = 'salmon')
plt.title('Transaction Types')
plt.xlabel('Type of Transaction')
plt.ylabel('Count')
plt.show()
```

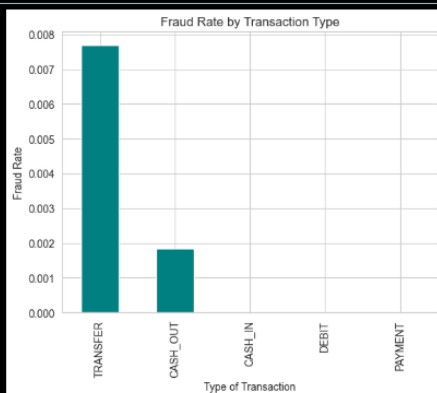
✓ 0.9s



Value Counts of the instances a type of transaction occurs.

```
fraud_by_type = df.groupby('type')['isFraud'].mean().sort_values(ascending=False)
fraud_by_type.plot(kind='bar', color='teal')
plt.title('Fraud Rate by Transaction Type')
plt.xlabel('Type of Transaction')
plt.ylabel('Fraud Rate')
plt.show()
```

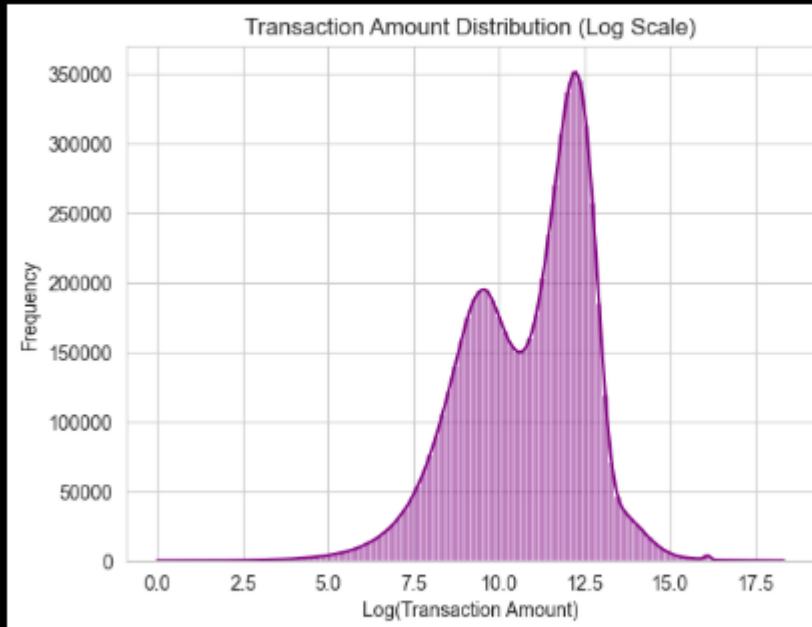
✓ 0.7s



Value counts of fraudulent transactions by type. Most of the fraudulent transactions occur during Transfer. Some Cash Out transactions are fraud and none of the Cash In, Debit and Payment transactions have been fraud.

```
sns.histplot(np.log1p(df['amount']), bins=100, color='purple', kde=True)
plt.title('Transaction Amount Distribution (Log Scale)')
plt.xlabel('Log(Transaction Amount)')
plt.ylabel('Frequency')
plt.show()
```

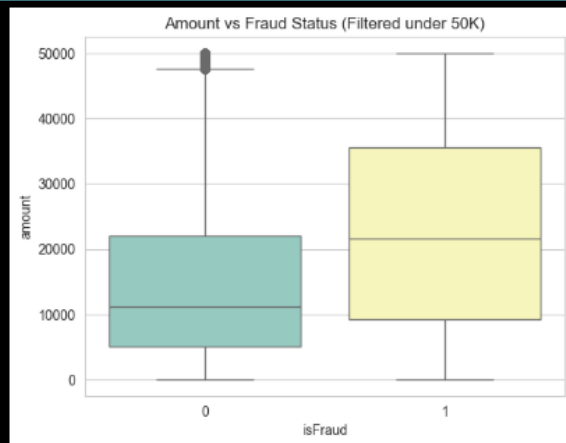
✓ 43.2s



Shows the Amount Log and Frequency of those transactions.

```
sns.boxplot(data = df[df['amount'] < 50000], x = 'isFraud', y = 'amount', palette = 'Set3')
plt.title('Amount vs Fraud Status (Filtered under 50K)')
plt.show()
```

✓ 12.0s



Generate Code Markdown

Median (middle value)

Fraud (1) has a higher median amount than Not Fraud (0). → This means fraudulent transactions tend to involve larger amounts (even in the under-50K range). Outliers

Non-fraud (0) shows a few outliers near 45–50K.

Fraud (1) might not show visible outliers (likely within whisker range).

```

frauds_per_step = df[df['isFraud'] == 1]['step'].value_counts().sort_index()
plt.plot(frauds_per_step.index, frauds_per_step.values, color='skyblue', label = 'Frauds')
plt.xlabel('Time Steps')
plt.ylabel('Number of Frauds')
plt.title('Number of Frauds Over Time Steps')
plt.grid(True)
plt.show()

```

✓ 0.1s



Shows the number of Fraud Transactions over periods of time.

```

fraud_types = df[df['type'].isin(['TRANSFER', 'CASH_OUT'])]

```

✓ 0.7s

```

fraud_types.head()

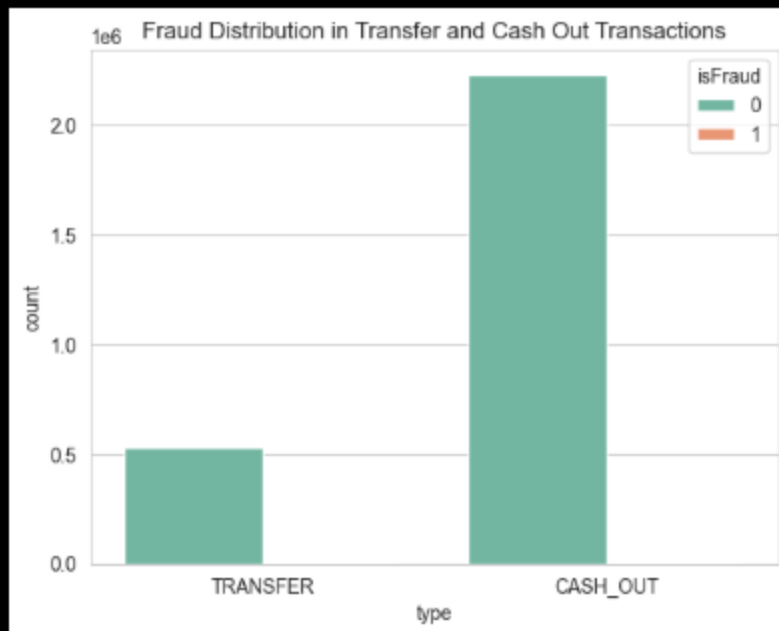
```

✓ 0.0s

	type	amount	nameOrig	oldbalanceOrig	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud	balanceDiffOrig	balanceDiffDest
2	TRANSFER	181.00	C1305486145	181.0	0.0	C553264065	0.0	0.00	1	0	181.0	0.00
3	CASH_OUT	181.00	C840083671	181.0	0.0	C38997010	21182.0	0.00	1	0	181.0	-21182.00
15	CASH_OUT	229133.94	C905080434	15325.0	0.0	C476402209	5083.0	51513.44	0	0	15325.0	46430.44
19	TRANSFER	215310.30	C1670993182	705.0	0.0	C1100439041	22425.0	0.00	0	0	705.0	-22425.00
24	TRANSFER	311685.89	C1984094095	10835.0	0.0	C932583850	6267.0	2719172.89	0	0	10835.0	2712905.89

```
sns.countplot(data = fraud_types, x = 'type', hue = 'isFraud', palette='Set2')
plt.title('Fraud Distribution in Transfer and Cash Out Transactions')
plt.show()
```

✓ 8.3s



Value Counts of Fraud types.

```
corr = df[['amount', 'oldbalanceOrig', 'newbalanceOrig', 'oldbalanceDest', 'newbalanceDest', 'isFraud']].corr()
```

✓ 2.8s

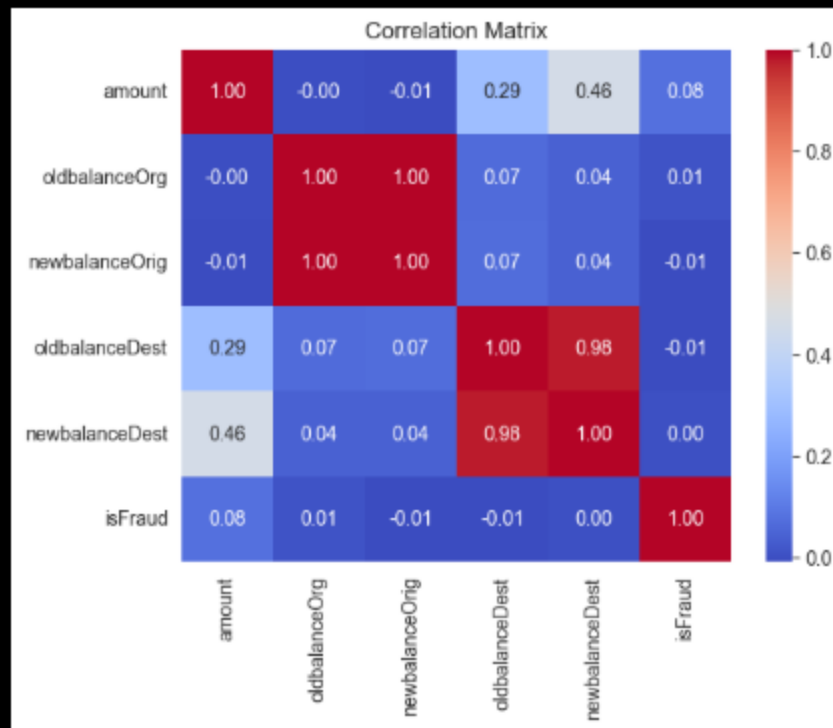
corr

✓ 0.0s

	amount	oldbalanceOrig	newbalanceOrig	oldbalanceDest	newbalanceDest	isFraud
amount	1.000000	-0.002762	-0.007861	0.294137	0.459304	0.076688
oldbalanceOrig	-0.002762	1.000000	0.998803	0.066243	0.042029	0.010154
newbalanceOrig	-0.007861	0.998803	1.000000	0.067812	0.041837	-0.008148
oldbalanceDest	0.294137	0.066243	0.067812	1.000000	0.976569	-0.005885
newbalanceDest	0.459304	0.042029	0.041837	0.976569	1.000000	0.000535
isFraud	0.076688	0.010154	-0.008148	-0.005885	0.000535	1.000000

```
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix')
plt.show()
```

✓ 0.4s



Correlation of all numerical features.

FEATURE SELECTION

```
print(classification_report(y_test, y_pred))
```

✓ 0.5s

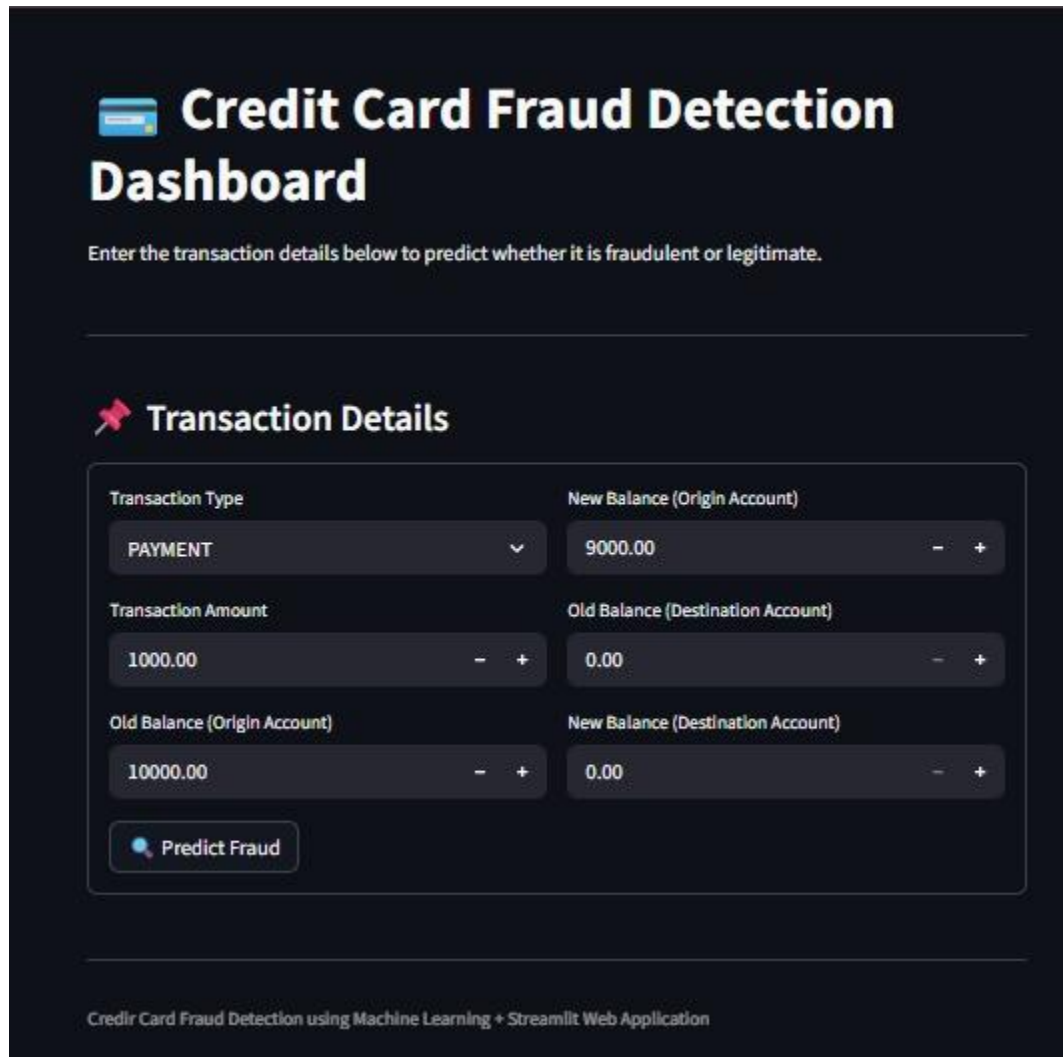
	precision	recall	f1-score	support
0	1.00	0.95	0.97	1906322
1	0.02	0.94	0.04	2464
accuracy			0.95	1908786
macro avg	0.51	0.94	0.51	1908786
weighted avg	1.00	0.95	0.97	1908786

```
confusion_matrix(y_test, y_pred)
```

✓ 0.1s

```
array([[1804823, 101499],
       [    151,   2313]])
```

DASHBOARD

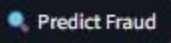


Credit Card Fraud Detection Dashboard

Enter the transaction details below to predict whether it is fraudulent or legitimate.

Transaction Details

Transaction Type	New Balance (Origin Account)
PAYMENT	9000.00
Transaction Amount	Old Balance (Destination Account)
1000.00	0.00
Old Balance (Origin Account)	New Balance (Destination Account)
10000.00	0.00

 Predict Fraud

Credit Card Fraud Detection using Machine Learning + Streamlit Web Application

Credit Card Fraud Detection Dashboard with default fields selected as example. This is a Legitimate Transaction.

Credit Card Fraud Detection Dashboard

Enter the transaction details below to predict whether it is fraudulent or legitimate.

Transaction Details

Transaction Type	New Balance (Origin Account)
PAYMENT	9000.00
Transaction Amount	Old Balance (Destination Account)
1000.00	0.00
Old Balance (Origin Account)	New Balance (Destination Account)
10000.00	0.00
<button>Predict Fraud</button>	

Prediction Result

✓ Transaction is Legitimate

Fraud Probability: 0.0000

Fraud Risk Level

Low Risk

Input Summary

	type	amount	oldbalanceOrig	newbalanceOrig	oldbalanceDest	newbalanceDest
0	PAYMENT	1000	10000	9000	0	0

LEGITIMATE TRANSACTION

 **Credit Card Fraud Detection Dashboard**

Enter the transaction details below to predict whether it is fraudulent or legitimate.

 **Transaction Details**

Transaction Type
PAYMENT

New Balance (Origin Account)
4880.00

Transaction Amount
120.00

Old Balance (Destination Account)
0.00

Old Balance (Origin Account)
5000.00

New Balance (Destination Account)
0.00

Predict Fraud

 **Prediction Result**

✓ Transaction is Legitimate

Fraud Probability: 0.0000

 **Fraud Risk Level**

Low Risk

Input Summary

	type	amount	oldbalanceOrig	newbalanceOrig	oldbalanceDest	newbalanceDest
0	PAYMENT	120	5000	4880	0	0

FRAUDULENT TRANSACTION



Credit Card Fraud Detection Dashboard

Enter the transaction details below to predict whether it is fraudulent or legitimate.

Transaction Details

Transaction Type
TRANSFER

New Balance (Origin Account)
8000.00

Transaction Amount
7500.00

Old Balance (Destination Account)
0.00

Old Balance (Origin Account)
8000.00

New Balance (Destination Account)
0.00

Predict Fraud

Prediction Result

Fraud Detected!

Fraud Probability: 0.1474

Fraud Risk Level

High Risk

Input Summary

	type	amount	oldbalanceOrig	newbalanceOrig	oldbalanceDest	newbalanceDest
0	TRANSFER	7500	8000	8000	0	0

Conclusion

The project successfully delivered a working Fraud Detection System. While the dataset was highly imbalanced, the use of **Logistic Regression with Balanced Weights** yielded a **94% Recall rate**. This means the system successfully catches 94 out of every 100 fraudulent transactions. We successfully integrated this model into a Streamlit web application, demonstrating how Machine Learning can be deployed in real-world banking scenarios to prevent financial loss.

References (Use IEEE Referencing style)

E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, "PaySim: A financial mobile money simulator for fraud detection," in 28th European Modeling and Simulation Symposium (EMSS), Larnaca, Cyprus, 2016.

Scikit-learn Developers, "Logistic Regression - scikit-learn documentation," [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html. [Accessed: 2023].

Streamlit Inc., "Streamlit Documentation," [Online]. Available: <https://docs.streamlit.io/>. [Accessed: 2023].

W. McKinney, "Data Structures for Statistical Computing in Python," in Proceedings of the 9th Python in Science Conference, 2010, pp. 56–61.

N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002. (Relevant if discussing imbalance handling techniques in literature review).

